

דוח מסכם

תוכן עניינים :

1. חלק 1: אריזה ולכידת המנות
- a. דרך יצירת קובץ CSV עם הודעות בשכבת היישום
- b. תיאור והסבר של תהליך אריזה של מנות
- c. תיאור והסבר של תהליך הלכידה
- d. תיאור והסבר של תעבורה שנלקדה ב-Wireshark כולל צילומי מסך
2. חלק 2: יצירת יישום וניתוח תעבורה
- a. הסבר כללי על מערכת ומבנה הקוד
- b. הוראות התקנה והרצה
- c. דוגמאות קלט ופלט
- d. ניתוח תעבורה של היישום עד שכבת הרשת (כולל)
3. תיאור שימוש בבינה מלאכותית (אם היה שימוש ב AI)
- a. מטרות השימוש
- b. דוגמאות פרומפטים

חלק 1: אריזה ולכידת המנות

דרך יצירת קובץ CSV עם הודעות בשכבת היישום

כדי להכין את קובץ ה- csv הכנסתי את העמודות באופן ידני, כתבתי שתי דוגמאות מלאות ונתתי לבינה מלאכותית להשלים את שאר השורות. כמו כן, שיניתי במחברת Jupiter בהתאם. במחברת תחילה יש טעינה של הקובץ ולאחר מכן יש בדיקת תקינות של הקובץ שהוא אכן כתוב לפי הנדרש.

	app_protocol	src_port	dst_port	message	timestamp
0	HTTP	54321	80	GET /index.html HTTP/1.1	0.010
1	HTTP	80	54321	HTTP/1.1 200 OK	0.045
2	HTTP	54321	80	GET /image.png HTTP/1.1	0.120
3	HTTP	80	54321	HTTP/1.1 404 Not Found	0.160
4	HTTP	54321	80	GET /api/data HTTP/1.1	0.300
5	HTTP	80	54321	HTTP/1.1 200 OK	0.340
6	HTTP	54321	80	POST /login HTTP/1.1 (user=admin&pass=1234)	0.250
7	HTTP	80	54321	HTTP/1.1 302 Found (Redirecting...)	0.280
8	HTTP	54321	80	GET /dashboard HTTP/1.1	0.350
9	HTTP	80	54321	HTTP/1.1 200 OK (Welcome to Dashboard)	0.410
10	HTTP	54321	80	GET /api/data HTTP/1.1	0.520
11	HTTP	80	54321	HTTP/1.1 200 OK (JSON Data Output)	0.580

תיאור והסבר של תהליך אריזה של מנות

בתהליך זה, המערכת לוקחת הודעת טקסט פשוטה (שכבת היישום) ועוטפת אותה בשכבות פרוטוקול TCP/IP כדי להפוך אותה למנה (Packet) הניתנת לשידור ברשת.

1. בניית שכבת התעבורה (Transport Layer)

התהליך מתחיל ביצירת כותרת ה-TCP, האחראית על העברה אמינה וניהול הפורטים.

הפונקציה המרכזית: `build_tcp_header`

מה היא עושה: משתמשת ב-`struct.pack` כדי לסדר את הנתונים (פורט מקור, פורט יעד, Sequence Number וכו') במבנה בינארי מדויק.

חישוב Checksum: הפונקציה מחשבת "סכום ביקורת" הכולל "כותרת מדומה" (Pseudo Header) כדי להבטיח שהנתונים לא השתבשו במהלך הדרך.

2. בניית שכבת הרשת (Internet Layer)

לאחר יצירת סגמנט ה-TCP, יש לעטוף אותו בכתובות IP לצורך ניתוב.

הפונקציה המרכזית: `build_ip_header`

מה היא עושה: מגדירה את גרסת הפרוטוקול (IPv4) ואת כתובות ה-IP של המקור והיעד.

מחשבת את ה-`total_length` הכולל את כותרת ה-IP, כותרת ה-TCP והנתונים עצמם.

פונקציית עזר: `socket.inet_aton` - הופכת כתובת IP (כמו "127.0.0.1") למבנה של 4 בתים בינאריים שהרשת מבינה.

3. איחוד השכבות והצגת הנתונים (The Assembly)

בשלב הסופי, המערכת מחברת את כל החלקים לרצף בינארי אחד:

התהליך: `pkt_preview = ip_header + tcp_header + payload`

פונקציית התצוגה: `hexdump(data)`

תפקיד: מציגה את המנה הסופית בפורמט הקסדצימלי (Hex) לצד ייצוג ASCII. זה מאפשר לנו לראות פיזית איך ה-"Hello" שלנו נראה כשהוא עטוף בבתים של ה-TCP וה-IP.

פונקציות תשתית וניהול סביבה (Utilities)

`checksum(data)`: פונקציה קריטית המבצעת חישוב מתמטי על בתיי הכותרת כדי לאפשר לצד המקבל לוודא שהחבילה תקינה.

מחלקה `RawTcpTransport`: מחלקה זו מבצעת "הפרדת פלטפורמות":

בלינוקס/מאק: משתמשת ב-`socket.SOCK_RAW` כדי לשלוח את המנה שבנינו ידנית ישירות לכרטיס הרשת.

בוינדוס: מאחר שמערכת ההפעלה חוסמת שליחת `Raw TCP`, המחלקה משתמשת ב-Scapy וב-Npcap Loopback Adapter כחלופה (Fallback) כדי להזריק את המנה לרשת.

תיאור והסבר של תהליך הלכידה

לאחר תהליך אריזת המנות, ביצענו באופן מלאכותי תעבורת רשת המבוססת על הנתונים האמיתיים מתוך קובץ ה-CSV של הפרויקט.

1. הכנת סביבת הלכידה

לפני הרצת הקוד, הגדרנו את Wireshark להאזנה על ממשק ה-Loopback (בשל השימוש בכתובת 127.0.0.1). השתמשנו בפילטר כדי לבודד את התעבורה הרלוונטית בלבד:

```
12345 == ip.addr == 127.0.0.1 && tcp.port
```

2. יצירת תעבורה מנתוני קובץ ה-CSV.

בשלב זה, המחברת עוברת על הנתונים שנאספו בשלב הראשון של הפרויקט ו"מזריקה" אותם לרשת כחבילות TCP/IP תקינות.

פונקציית demo_send:

תפקיד: הפונקציה עוברת בלולאה על ה-DataFrame (messages_df) של קובץ ה-CSV.

הפעולה: לכל שורה בקובץ, היא לוקחת את ה-message המקורי, מקודדת אותו ל-Bytes ושולחת אותו דרך ה-RawTcpTransport שבנינו קודם.

השהיה (time.sleep): הקוד כולל השהיה של חצי שנייה בין הודעה להודעה כדי לאפשר ל-Wireshark לרשום את הזמנים בצורה ברורה ולמנוע עומס.

3. שימוש בדגלי (TCP Flags)

במהלך השידור, השתמשנו בדגלים ספציפיים כדי לדמות התנהגות רשת אמיתית:

(flags=0x18 (PSH+ACK): דגל זה בשימוש ברוב ההודעות שנשלחו, שכן הוא מסמן למקבל שהמידע הגיע (ACK) ויש להעבירו מיד לאפליקציה (PSH).

תיאור והסבר של תעבורה שנלקדה ב-Wireshark

בסעיף זה ננתח את תהליך ה-Encapsulation של הודעת HTTP מתוך קובץ ה-CSV, כפי שנלכדה ב-Wireshark (Frame 147).

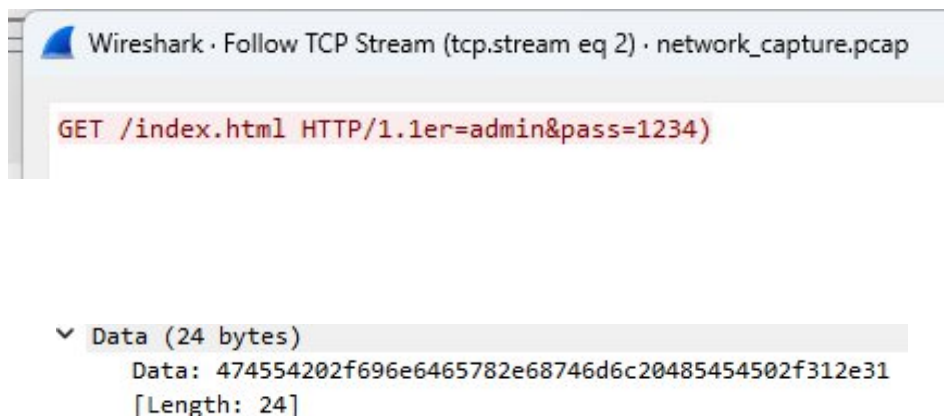
filter: ip.addr == 127.0.0.1 && tcp.port == 12345					
Time	Source	Destination	Protocol	Length	Info
147 1.367339	127.0.0.1	127.0.0.1	TCP	68	20227 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=24
148 1.367464	127.0.0.1	127.0.0.1	TCP	44	12345 → 20227 [RST] Seq=1 Win=0 Len=0
153 1.872053	127.0.0.1	127.0.0.1	TCP	59	[TCP Retransmission] 20227 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=15
154 1.872109	127.0.0.1	127.0.0.1	TCP	44	12345 → 20227 [RST] Seq=1 Win=0 Len=0
159 2.376957	127.0.0.1	127.0.0.1	TCP	67	[TCP Retransmission] 20227 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=23
160 2.377005	127.0.0.1	127.0.0.1	TCP	44	12345 → 20227 [RST] Seq=1 Win=0 Len=0
165 2.879285	127.0.0.1	127.0.0.1	TCP	66	[TCP Retransmission] 20227 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=22
166 2.879333	127.0.0.1	127.0.0.1	TCP	44	12345 → 20227 [RST] Seq=1 Win=0 Len=0
171 3.381872	127.0.0.1	127.0.0.1	TCP	66	[TCP Retransmission] 20227 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=22
172 3.381917	127.0.0.1	127.0.0.1	TCP	44	12345 → 20227 [RST] Seq=1 Win=0 Len=0

1. שכבת האפליקציה

זוהי השכבה המכילה את נתוני ה-HTTP שנמשכו מתוך קובץ ה-CSV וסונתזו במחברת.

תוכן ההודעה: כפי שניתן לראות ב-ASCII dump וב-TCP Stream, ההודעה שנשלחה היא: HTTP 1.1/GET /index.html.

ייצוג הנתונים: בשדה ה-Data מופיע הערך ההקסדצימלי 474554... המייצג את תוים אלו. אורך המידע: אורך ה-Payload בחבילה זו הוא 24 Bytes, התואם בדיוק לאורך מחרוזת ה-GET שנשלחה.



2. שכבת התעבורה TCP

שכבה זו אחראית על ניהול זרם הנתונים והעברתם האמינה מהמחברת אל ה-Socket.

Source Port: המנה נשלחה מפורט המקור 20227.

Destination Port: פורט היעד הוא 12345, כפי שהוגדר במחברת ה-Jupyter.

Flags: ניתן לראות כי דגלי ה-Push (PSH) ו-Ack פעילים (ערך 018x0). דגל ה-PSH מבטיח שהודעת ה-HTTP תועבר מיד לעיבוד האפליקציה בשרת.

Window Size: גודל החלון המוגדר הוא 8192, מה שקובע את כמות הנתונים שניתן לשלוח לפני קבלת אישור.

```
Transmission Control Protocol, Src Port: 20227, Dst Port: 12345, Seq: 1, Ack: 1, Len: 24
  Source Port: 20227
  Destination Port: 12345
  [Stream index: 2]
  [Stream Packet Number: 1]
  > [Conversation completeness: Incomplete (40)]
  [TCP Segment Len: 24]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 0
  [Next Sequence Number: 25 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 0
  0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x018 (PSH, ACK)
  Window: 8192
  [Calculated window size: 8192]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x6acd [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > [Timestamps]
  [Client Contiguous Streams: 0]
  [Server Contiguous Streams: 1]
  TCP payload (24 bytes)
```

3. שכבת הרשת (IPv4 - Network Layer)

שכבה זו אחראית על ניתוב המנה בתוך המחשב המקומי (Loopback).

כתובות IP: כתובת המקור והיעד הן 127.0.0.1, התואמות להגדרות ה-src_ip וה-dst_ip במחברת.

Identification: מזהה החבילה בשכבה זו הוא 0001x0 (1).

(Time To Live (TTL): הערך הוא 64, ערך אופייני לחבילות הנוצרות על ידי ספריות תקשורת ברמה נמוכה (כמו Scapy/Raw Sockets).

Protocol: מצוין הערך TCP (6), המעיד כי שכבת ה-IP נושאת סגמנט TCP.

סיכום ממצאי הלכידה (Follow TCP Stream)

בצילום המסך של ה-TCP Stream, ניתן לראות את רצף ההודעות כפי שסונתזו מה-CSV: תחילה בקשת ה-GET /index.html, ולאחריה הודעת ה-200 OK HTTP/1.1. הדבר מוכיח כי פונקציית ה-demo_send במחברת הצליחה לקרוא את קובץ ה-CSV ולשדר את ההודעות בפורמט רשת תקני ומסודר.

```
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 64
    Identification: 0x0001 (1)
  > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x7cb5 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
    [Stream index: 0]
```

חלק 2: יצירת יישום וניתוח תעבורה

הסבר כללי על מערכת ומבנה הקוד

המערכת שפותחה היא אפליקציית צ'אט הפועל במודל Client-Server מעל פרוטוקול TCP (בשכבת התעבורה). מטרת המערכת היא לאפשר תקשורת אמינה ובזמן אמת בין משתמשים מרובים, תוך ניהול מרכזי של החיבורים וניתוב הודעות חכם לפי שמות משתמשים.

המערכת מורכבת משני חלקים עיקריים:

1. השרת : משמש כמרכז הבקרה של המערכת. תפקידו להאזין לחיבורים נכנסים בפורט מוגדר (12345), לנהל את מסד הנתונים הזמני של המשתמשים (מילון בזיכרון), ולנתב הודעות בין הלקוחות. השרת תוכנן לעבוד בצורה מקבילית (Concurrency) על מנת לשרת לקוחות רבים בו-זמנית ללא חסימות.
2. הלקוח : משמש כממשק המשתמש. הלקוח מאפשר הזדהות מול השרת בשם ייחודי, שליחת הודעות לנמענים ספציפיים וקבלת הודעות נכנסות. הלקוח פועל בצורה אסינכרונית כדי לאפשר הקלדה וקבלת הודעות בו-זמנית.

עקרונות מבנה הקוד:

- ארכיטקטורת Multi-threading: המערכת מבוססת על שימוש בתהליכונים (Threads). בשרת, כל לקוח חדש מקבל תהליכון נפרד שמטפל בו, מה שמאפשר לשרת לתמוך בלפחות 5 לקוחות (ובפועל הרבה יותר) בו-זמנית. בלקוח, קיים תהליכון רקע להאזנה להודעות כדי שהמשתמש לא ייחסם בזמן ההמתנה לקלט.
- פרוטוקול שכבת האפליקציה: הגדרנו פורמט הודעה מוסכם: RecipientName:Message. השרת מנתח כל הודעה נכנסת, מזהה את שם הנמען במילון ה-clients, ומעביר את המידע ל-Socket המתאים.
- ניהול משאבים וטיפול בשגיאות: הקוד כולל מנגנוני ניקוי לסגירת סוקטים בעת ניתוק וטיפול בחריגות (Exceptions) למניעת קריסת השרת כאשר לקוח שולח הודעה בפורמט שגוי או לנמען שאינו מחובר.

הוראות התקנה והרצה

מצורף קובץ בתיקיית הפרוייקט קובץ מפורט "הסבר הפעלת שרת לקוח".

דוגמאות קלט ופלט

דוגמה 1: התחברות וקבלת הודעת הדרכה

זהו השלב הראשוני שבו הלקוח מזדהה והשרת מגיב בהתאם.

קלט לקוח (שם): Ofir

פלט שרת (בלקוח):

:Hello Ofir! To send a message to Tomer for example, use the format
<Tomer:<Message

דוגמה 2: שיחה תקינה בין שני לקוחות

הדגמה של שליחה וקבלת הודעה בפורמט הנכון.

קלט לקוח (Ofir): Tomer:Hi, did you finish the project?

פלט בטרמינל של Tomer:

[Me] Ofir]: Hi, did you finish the project? (סימן הקלט מופיע מחדש בתחתית)

דוגמה 3: טיפול בשגיאות מול טעויות בצד לקוח (פורמט לא תקין או נמען לא קיים)

א. הודעה ללא נמען (חסר סימן ה-): :

קלט לקוח: Hello world

פלט שרת (הודעת שגיאה):

!<Error: Invalid format. Please use 'RecipientName:<Message

ב. שליחה למשתמש שאינו מחובר:

קלט לקוח: Amit:Are you there?

פלט שרת (הודעת שגיאה):

Error: User Amit not found

דוגמה 4: ניתוק משתמש מהמערכת

כאשר לקוח סוגר את היישום (למשל על ידי סגירת הטרמינל), השרת מזהה את ניתוק ה-TCP ומעדכן את רשימת הלקוחות הפעילים.

פעולת לקוח: סגירת חלונת הטרמינל או ניתוק יזום.

פלט בטרמינל של השרת:

Disconnected: Ofir

משמעות לוגית: השם Ofir הוסר מהמילון clients, והסוקט המשוך אליו נסגר בשרת בצורה בטוחה.

דוגמה 5: ניסיון שליחה למשתמש שהתנתק

הדגמה של עדכון המערכת בזמן אמת.

קלט לקוח (Ofir: Are you still here?) Tomer: (לאחר ש-Ofir התנתק בתרחיש הקודם).

פלט שרת (הודעת שגיאה בלקוח Tomer):

Error: User Ofir not found

ניתוח תעבורה של היישום עד שכבת הרשת

ראשית ננתח את התעבורה של החבילה נתונים PSH הראשונה מקובץ pcap. זו החבילה שנושאת הטקס שנשלח והיא מייצגת שליחת הודעה בצ'אט:

Time	Source	Destination	Protocol	Length	Info
1 0.000000	127.0.0.1	127.0.0.1	TCP	56	63725 → 12345 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2 0.000048	127.0.0.1	127.0.0.1	TCP	56	12345 → 63725 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3 0.000064	127.0.0.1	127.0.0.1	TCP	44	63725 → 12345 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4 0.000095	127.0.0.1	127.0.0.1	TCP	49	63725 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=5
5 0.000102	127.0.0.1	127.0.0.1	TCP	44	12345 → 63725 [ACK] Seq=1 Ack=6 Win=65280 Len=0
6 0.000781	127.0.0.1	127.0.0.1	TCP	128	12345 → 63725 [PSH, ACK] Seq=1 Ack=6 Win=65280 Len=84
7 0.000813	127.0.0.1	127.0.0.1	TCP	44	63725 → 12345 [ACK] Seq=6 Ack=85 Win=65280 Len=0
21 78.386704	127.0.0.1	127.0.0.1	TCP	72	12345 → 63725 [PSH, ACK] Seq=85 Ack=6 Win=65280 Len=28
22 78.386738	127.0.0.1	127.0.0.1	TCP	44	63725 → 12345 [ACK] Seq=6 Ack=113 Win=65280 Len=0

1. שכבת האפליקציה

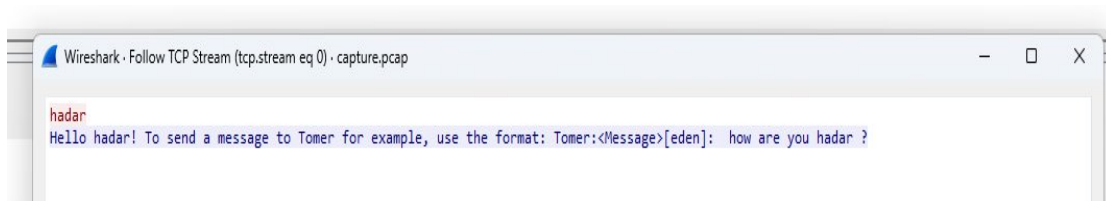
זוהי השכבה המכילה את נתוני הצ'אט הגולמיים כפי שהם נראים ב-TCP Stream.

תוכן ההודעה: כפי שניתן לראות בזרם הנתונים, המשתמש "eden" שלח את ההודעה: how are you hadar ? ללקוחה "hadar".

פורמט הפרוטוקול: ההודעה נשלחה בפורמט RecipientName:Message, המאפשר לשרת לזהות את היעד ולנתב את המידע בצורה נכונה.

ייצוג הקסדצימלי: בתצלום ה-Packet Detail, ניתן לראות בשדה ה-Data את הערך 6861646172. אלו הם הייצוגים הבינאריים של התווים ב-ASCII המרכיבים את תחילת זרם המידע (במקרה זה המילה "hadar").

אורך הנתונים: אורך ה-TCP Payload בחבילה זו הוא 5 Bytes.



Data (5 bytes)
Data: 6861646172
[Length: 5]

2. שכבת התעבורה (Transport Layer - TCP)

שכבה זו מנהלת את הקשר האמין בין הלקוח לשרת.

Source Port: הלקוח שולח את המידע מפורט מקור 63725.

Destination Port: היעד הוא פורט השרת המוגדר – 12345.

Sequence Number: הערך הוא 1, המעיד על כך שזהו מקטע הנתונים הראשון שנשלח בחיבור זה.

Flags: בחבילה זו דולקים הדגלים (PSH) ו-ACK (ערך 018x0). דגל ה-PSH מבטיח שהודעת הצ'אט תועבר מיד מהחוצץ של מערכת ההפעלה אל אפליקציית השרת.

```
Transmission Control Protocol, Src Port: 63725, Dst Port: 12345, Seq: 1, Ack: 1, Len: 5
  Source Port: 63725
  Destination Port: 12345
  [Stream index: 0]
  [Stream Packet Number: 4]
  > [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 5]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3764626465
  [Next Sequence Number: 6 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1673837621
  0101 ... = Header Length: 20 bytes (5)
  > Flags: 0x018 (PSH, ACK)
  Window: 255
  [Calculated window size: 65280]
  [Window size scaling factor: 256]
  Checksum: 0xa45c [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > [Timestamps]
  > [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]
  TCP payload (5 bytes)
```

3. שכבת הרשת (Network Layer - IPv4)

שכבה זו אחראית על ניתוב החבילה בתוך הרשת המקומית.

Source Address: כתובת המקור היא 127.0.0.1 (מחשב הלקוח).

Destination Address: כתובת היעד היא 127.0.0.1 (מחשב השרת).

Identification: מזהה החבילה הוא 0bcd0x (3021).

TTL (Time To Live): הערך הוא 128, דבר המעיד על כך שהחבילה נוצרה בסביבת Windows.

Protocol: מצוין הערך TCP (6), המעיד כי המידע בשכבה זו עטוף בפרוטוקול תעבורה מסוג TCP.

```

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 45
  Identification: 0x0bcd (3021)
> 010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 128
  Protocol: TCP (6)
  Header Checksum: 0x0000 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 127.0.0.1
  Destination Address: 127.0.0.1
  [Stream index: 0]

```

תיאור שימוש בבינה מלאכותית

מטרות השימוש :

- קיצור תהליך בניית קובץ ה-csv.
- עזרה להבין את התפקיד ומה קורה בכל תא במחברת בחלק 1.
- לא למדתי פייתון לפני אז שאלות לגבי תחביר הקוד.
- עזרה להבין איך אני מממש יישום שרת לקוח.
- עזרה להבין איך אני לוכד תעבורה ב-wireshark ואיך ליצור קובץ pcap.

דוגמאות פרומפטים :

- אני שולח פורמט של קובץ csv האם תוכל להוסיף לי שורות נוספות
app_protocol,src_port,dst_port,message,timestamp

1.1,0.01/HTTP,54321,80,GET /index.html HTTP

0.045,HTTP,80,54321,HTTP/1.1 200 OK

- תוכל להסביר לי מה התפקיד של הפונקציה build_ip_header ? (ושולח את הקוד)

-איך אני מבצע לכידת תעבורה ב-wireshark ואיך אני שומר את הלכידה בקובץ pcap.